

Campus Finder

Important SQL Queries

The queries to demonstrate and explain, with a join count, a line-by-line explanation, sample output and likely questions for each.

ALL QUERIES AT A GLANCE

#	QUERY	JOINS	JOIN TYPES	KEY CONCEPTS
Q1	Report an item	0	none (INSERT)	INSERT, Sequence, PK / FK / CHECK
Q2	Search items by category and status	3	3 INNER	INNER JOIN, PK-FK join, WHERE
Q3	Possible match finder	1	1 self-join (INNER, non-equi)	Self-join, CASE, Weighted scoring
Q4	Possible matches with names (best only)	6	6 INNER (1 of them a self-join)	Multi-table join, Same table, two aliases, Inline view
Q5	My Claims for one student	3	3 INNER	Join chain, Weak entity, Composite primary key
Q6	Submit a claim, approve one, reject the rest	0	none (DML)	INSERT, UPDATE, DEFAULT
Q7	Reports per category (GROUP BY and HAVING)	1	1 INNER	Aggregates, GROUP BY, WHERE vs HAVING
Q8a	Found items nobody has claimed (outer join)	2	1 INNER + 1 LEFT OUTER	LEFT OUTER JOIN, Anti-join, NULL handling
Q8b	Same result using a subquery (NOT EXISTS)	1	1 INNER + 1 correlated subquery	Correlated subquery, NOT EXISTS, Join vs subquery
Q9	Top 3 reporters	1	1 INNER	JOIN + GROUP BY, Top-N, Tie handling
Q10	Activity summary across all six tables	5	3 INNER + 2 LEFT OUTER	Six-table join, Outer joins, Row multiplication

How joins are counted. Joins = the number of JOIN operations = (table instances in FROM) – 1.

A table joined twice under different aliases counts twice (Q4 uses ITEM, STUDENT and LOCATION two times each). A subquery such as NOT EXISTS is not a join. In the SQL listings each JOIN line is highlighted and numbered.

THE SIX TABLES

TABLE	KIND	PRIMARY KEY	FOREIGN KEYS
STUDENT	strong	student_id	—
CATEGORY	strong	category_id	—
LOCATION	strong	location_id	—
ITEM	strong	item_id	student_id, category_id, location_id
CLAIM	weak (depends on ITEM)	item_id + student_id	item_id, student_id
ITEM_IMAGE	weak (depends on ITEM)	item_id + image_no	item_id

Relationships: STUDENT, CATEGORY and LOCATION each have a 1:N relationship with ITEM; ITEM has a 1:N relationship with CLAIM and with ITEM_IMAGE. Data types and constraints are in [docs/campus_finder_queries.sql](#).

Used on: Report page (ReportItemServlet)

Saves one lost or found report. The form fields map one-to-one onto the columns of ITEM.

```
INSERT INTO item (item_id, student_id, category_id, location_id,
                 item_name, description, item_status, reported_date)
VALUES (item_seq.NEXTVAL, 5004, 101, 201,
       'Silver Laptop Charger', 'Silver laptop charger left near the library desk',
       'FOUND', DATE '2026-10-07');

SELECT item_id, item_name, item_status, reported_date
FROM   item
WHERE  item_id = (SELECT MAX(item_id) FROM item);

ROLLBACK;
```

JOINS IN THIS QUERY: 0 (none (INSERT))

An INSERT writes to a single table, so there is nothing to join.

HOW TO EXPLAIN IT

- `item_seq.NEXTVAL` asks the sequence for the next unique `item_id`, so no one numbers items by hand.
- The form sends `item_status` (LOST/FOUND), `item_name`, `category_id`, `location_id` and `description`; the new date field fills `reported_date`. The servlet supplies the reporting student's ID.
- Constraints protect the data: the foreign keys reject an unknown student, category or location, and the CHECK constraint allows only LOST or FOUND.
- In the Java servlet this should run as a `PreparedStatement` with `?` placeholders, which blocks SQL injection.
- The script ends with `ROLLBACK`, so the demo row is undone and the sample data stays unchanged.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	ITEM_STATUS	REPORTED_DATE
1013	Silver Laptop Charger	FOUND	07-OCT-26

1 row selected

They may ask: Why use a sequence instead of `MAX(item_id) + 1`?

Answer: Two users reporting at the same moment would both read the same maximum and collide on the primary key. A sequence hands each insert a different number atomically.

INSERT

Sequence

PK / FK / CHECK

PreparedStatement

Transaction

Q2 Search items by category and status

3 joins

Used on: Search page

Lists reported items with readable names, filtered the way the Search page lets a student filter (category, location, status).

```
SELECT i.item_id,
       i.item_name,
       i.item_status,
       c.category_name,
       l.location_name,
       i.reported_date,
       s.student_name AS reported_by
FROM   item i
JOIN   category c ON c.category_id = i.category_id
JOIN   location l ON l.location_id = i.location_id
JOIN   student s ON s.student_id = i.student_id
WHERE  i.item_status = 'FOUND'
AND    i.category_id = 103
ORDER BY i.reported_date DESC;
```

JOIN 1

JOIN 2

JOIN 3

JOINS IN THIS QUERY: 3 (3 INNER)

#	TYPE	TABLES	CONDITION
1	INNER	ITEM i ↔ CATEGORY c	c.category_id = i.category_id
2	INNER	ITEM i ↔ LOCATION l	l.location_id = i.location_id
3	INNER	ITEM i ↔ STUDENT s	s.student_id = i.student_id

Every join is primary key = foreign key. Because those foreign keys are NOT NULL, each item matches exactly one row in each table, so no rows are lost or duplicated.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	ITEM_STATUS	CATEGORY_NAME	LOCATION_NAME	REPORTED_DATE	REPORTED_BY
1012	Umbrella	FOUND	Accessories	Main Block	06-OCT-26	Diya Patel
1011	Steel Water Bottle	FOUND	Accessories	Main Block	04-OCT-26	Kabir Singh
1009	Brown Wallet	FOUND	Accessories	Canteen	02-OCT-26	Meera Nair

3 rows selected

HOW TO EXPLAIN IT

- ITEM stores only IDs (category_id, location_id, student_id). The three joins bring back the names a person wants to read.
- The WHERE clause is the search filter. A servlet would add a condition only for the filters the student actually picked, using ? placeholders.
- ORDER BY reported_date DESC shows the newest reports first.

They may ask: Why not store the category name directly in ITEM?

Answer: The same name would be repeated on every row. Renaming a category would mean many updates and risks inconsistency (an update anomaly). Keeping it in CATEGORY, the table is in 3NF and the name changes in one place.

INNER JOIN

PK-FK join

WHERE

ORDER BY

Normalisation

Used on: Matches page · project slide 9 (the novelty feature)

Scores every LOST report against every FOUND report out of 100. This is the query from your presentation; only a tie-breaker was added to ORDER BY.

```
SELECT l.item_id AS lost_item_id,
       f.item_id AS found_item_id,
       ( CASE WHEN l.category_id = f.category_id THEN 30 ELSE 0 END
       + CASE WHEN l.location_id = f.location_id THEN 30 ELSE 0 END
       + CASE WHEN ABS(l.reported_date - f.reported_date) <= 3 THEN 20 ELSE 0 END
       + CASE WHEN UPPER(l.description) LIKE '%' || UPPER(f.item_name) || '%'
               THEN 20 ELSE 0 END
       ) AS match_score
FROM   item l
JOIN   item f ON l.item_id != f.item_id
WHERE  l.item_status = 'LOST'
AND    f.item_status = 'FOUND'
ORDER BY match_score DESC, l.item_id, f.item_id;
```

JOIN 1

JOINS IN THIS QUERY: 1 (1 self-join (INNER, non-equi))

#	TYPE	TABLES	CONDITION
1	INNER (self)	ITEM l ↔ ITEM f	<code>l.item_id != f.item_id</code>

The same table is used twice: **l** plays the lost report and **f** the found report. The join condition is not an equality, so it is a non-equi self-join.

HOW TO EXPLAIN IT

- The WHERE clause keeps only LOST on the **l** side and FOUND on the **f** side, so every lost report is paired with every found report (5 lost × 7 found = 35 rows).
- Four independent `CASE` expressions each return points and the `+` signs add them: same category 30, same location 30, within 3 days 20, description keyword 20. Maximum 100.
- `ABS(l.reported_date - f.reported_date)`: in Oracle, subtracting two DATES gives a number of days; ABS removes the sign so the order does not matter.
- `UPPER(l.description) LIKE '%' || UPPER(f.item_name) || '%'` checks whether the lost description contains the found item's name, ignoring case.
- The best matches come first. In the sample data the three genuine pairs score 100, 80 and 70.

SAMPLE OUTPUT

LOST_ITEM_ID	FOUND_ITEM_ID	MATCH_SCORE
1001	1006	100
1002	1007	80
1003	1008	70
1001	1008	50
1003	1009	50
1004	1009	50
1004	1011	50
1005	1006	50

35 rows selected · first 8 shown

Tip: if `reported_date` ever stores a time (for example from `SYSDATE`), 3 calendar days can come out as 3.4 and fail `<= 3`. Wrap both dates in `TRUNC()`: `ABS(TRUNC(l.reported_date) - TRUNC(f.reported_date))`.

They may ask: Are the CASE expressions nested?

Answer: No. There are four CASE expressions placed side by side and added together; none is inside another. Together they form one score.

Self-join

CASE

Weighted scoring

UPPER / LIKE / ||

Date arithmetic

Used on: Matches page

What the Matches screen needs: Q3 plus readable names, a minimum score, and no student matched with their own report.

```
SELECT *
FROM (
  SELECT l.item_id      AS lost_item_id,
         l.item_name    AS lost_item,
         ls.student_name AS lost_by,
         ll.location_name AS lost_at,
         f.item_id      AS found_item_id,
         f.item_name    AS found_item,
         fs.student_name AS found_by,
         fl.location_name AS found_at,
         lc.category_name AS category,
         ( CASE WHEN l.category_id = f.category_id THEN 30 ELSE 0 END
         + CASE WHEN l.location_id = f.location_id THEN 30 ELSE 0 END
         + CASE WHEN ABS(l.reported_date - f.reported_date) <= 3 THEN 20 ELSE 0 END
         + CASE WHEN UPPER(l.description) LIKE '%' || UPPER(f.item_name) || '%'
           THEN 20 ELSE 0 END
         ) AS match_score
  FROM   item l
  JOIN   item f      ON l.item_id != f.item_id
  JOIN   student ls   ON ls.student_id = l.student_id
  JOIN   student fs   ON fs.student_id = f.student_id
  JOIN   category lc  ON lc.category_id = l.category_id
  JOIN   location ll  ON ll.location_id = l.location_id
  JOIN   location fl  ON fl.location_id = f.location_id
  WHERE  l.item_status = 'LOST'
  AND    f.item_status = 'FOUND'
  AND    l.student_id <> f.student_id
)
WHERE match_score >= 60
ORDER BY match_score DESC, lost_item_id;
```

JOIN 1

JOIN 2

JOIN 3

JOIN 4

JOIN 5

JOIN 6

JOINS IN THIS QUERY: 6 (6 INNER (1 of them a self-join))

#	TYPE	TABLES	CONDITION
1	INNER (self)	ITEM l ↔ ITEM f	<code>l.item_id != f.item_id</code>
2	INNER	STUDENT ls (lost reporter)	<code>ls.student_id = l.student_id</code>
3	INNER	STUDENT fs (found reporter)	<code>fs.student_id = f.student_id</code>
4	INNER	CATEGORY lc	<code>lc.category_id = l.category_id</code>
5	INNER	LOCATION ll (lost at)	<code>ll.location_id = l.location_id</code>
6	INNER	LOCATION fl (found at)	<code>fl.location_id = f.location_id</code>

Seven table instances, six joins. ITEM, STUDENT and LOCATION each appear twice under different aliases, once for each role.

SAMPLE OUTPUT

LOST_ITEM_ID	LOST_ITEM	LOST_BY	LOST_AT	FOUND_ITEM_ID	FOUND_ITEM	FOUND_BY	FOUND_AT	CATEGORY	MATCH_SCORE
1001	Black Earbuds	Aarav Sharma	Library	1006	Black Earbuds	Meera Nair	Library	Electronics	100
1002	ID Card	Diya Patel	Main Block	1007	ID Card	Aarav Sharma	Main Block	Documents	80
1003	Bike Keys	Rohan Mehta	Canteen	1008	Bike Keys	Diya Patel	Library	Keys	70

3 rows selected

They may ask: Why is the query wrapped in another SELECT?

Answer: WHERE is evaluated before the SELECT list, so it cannot see the alias `match_score`. Putting the scoring query in the FROM clause makes the alias a real column the outer WHERE can use.

Multi-table join

Same table, two aliases

Inline view

Threshold filter

HOW TO EXPLAIN IT

- The inner query is Q3 with six joins bolted on to turn IDs into names.
- It is wrapped in an **inline view** because a column alias such as `match_score` does not exist yet when WHERE runs. The outer query can filter on it.
- `match_score >= 60` hides weak pairs. 60 is a tunable choice: category and location alone already give 60.
- `l.student_id <> f.student_id` stops a student from being matched with something they found themselves.

Used on: My Claims page

Shows every claim a student has made, with the item it is about and the claim status.

```
SELECT c.item_id,
       i.item_name,
       cat.category_name,
       loc.location_name,
       c.claim_date,
       c.claim_status
FROM   claim c
JOIN   item i      ON i.item_id      = c.item_id
JOIN   category cat ON cat.category_id = i.category_id
JOIN   location loc ON loc.location_id = i.location_id
WHERE  c.student_id = 5005
ORDER BY c.claim_date DESC;
```

JOIN 1

JOIN 2

JOIN 3

JOINS IN THIS QUERY: 3 (3 INNER)

#	TYPE	TABLES	CONDITION
1	INNER	CLAIM c ↔ ITEM i	i.item_id = c.item_id
2	INNER	ITEM i ↔ CATEGORY cat	cat.category_id = i.category_id
3	INNER	ITEM i ↔ LOCATION loc	loc.location_id = i.location_id

A chain of joins: CLAIM → ITEM, then from ITEM out to CATEGORY and LOCATION.

HOW TO EXPLAIN IT

- CLAIM holds only `item_id`, `student_id`, the date, a description and the status, so it joins to ITEM to show what was claimed.
- `WHERE c.student_id = 5005` picks the student whose claims are shown (a `?` placeholder in the servlet).
- CLAIM is a **weak entity**: its primary key is the pair (`item_id`, `student_id`). A student can claim a given item only once; a second attempt fails with ORA-00001.
- Item 1008 has two claims from different students, which is the 1:N relationship from ITEM to CLAIM.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	CATEGORY_NAME	LOCATION_NAME	CLAIM_DATE	CLAIM_STATUS
1010	Notebook	Books	Parking Area	07-OCT-26	REJECTED
1008	Bike Keys	Keys	Library	02-OCT-26	PENDING

2 rows selected

They may ask: Why is CLAIM a weak entity?

Answer: A claim has no meaning without the item being claimed, so it borrows ITEM's key as part of its own composite primary key.

[Join chain](#)
[Weak entity](#)
[Composite primary key](#)
[Filter by user](#)

Q6 Submit a claim, approve one, reject the rest

0 joins

Used on: Claims workflow: student submits, staff decides

The claim life cycle in three statements, run as one transaction.

```
INSERT INTO claim (item_id, student_id, claim_date, claim_description)
VALUES (1011, 5003, DATE '2026-10-08', 'Silver bottle with a dent near the base');

UPDATE claim
SET   claim_status = 'APPROVED'
WHERE item_id = 1008
AND   student_id = 5003;

UPDATE claim
SET   claim_status = 'REJECTED'
WHERE item_id = 1008
AND   student_id <> 5003
AND   claim_status = 'PENDING';

SELECT item_id, student_id, claim_status
FROM   claim
WHERE  item_id IN (1008, 1011)
ORDER BY item_id, student_id;

ROLLBACK;
```

JOINS IN THIS QUERY: 0 (none (DML))

INSERT and UPDATE statements work on a single table and need no join.

HOW TO EXPLAIN IT

- The INSERT leaves `claim_status` out, so the `DEFAULT 'PENDING'` applies.
- The first UPDATE approves one student's claim on item 1008. The second rejects the other claims still pending on the same item.
- These changes belong together. `COMMIT` saves all of them, `ROLLBACK` undoes all of them. The script rolls back so the sample data is unchanged.
- The final SELECT shows the result: 1008/5003 approved, 1008/5005 rejected, 1011/5003 pending.

SAMPLE OUTPUT

ITEM_ID	STUDENT_ID	CLAIM_STATUS
1008	5003	APPROVED
1008	5005	REJECTED
1011	5003	PENDING

3 rows selected

They may ask: What stops two claims on the same item from both being approved?

Answer: At the moment only the transaction logic does. A function-based unique index such as `CREATE UNIQUE INDEX one_approved ON claim (CASE WHEN claim_status = 'APPROVED' THEN item_id END)` would enforce it in the database.

INSERT

UPDATE

DEFAULT

COMMIT / ROLLBACK

Atomicity (ACID)

Q7 Reports per category (GROUP BY and HAVING)

1 join

Used on: Admin / statistics report (extra)

Counts reports per category, split into lost and found, ignoring older reports and small groups.

```
SELECT c.category_name,
       COUNT(*) AS total_reports,
       SUM(CASE WHEN i.item_status = 'LOST' THEN 1 ELSE 0 END) AS lost,
       SUM(CASE WHEN i.item_status = 'FOUND' THEN 1 ELSE 0 END) AS found
FROM   item i
JOIN   category c ON c.category_id = i.category_id
WHERE  i.reported_date >= DATE '2026-09-29'
GROUP BY c.category_name
HAVING COUNT(*) >= 2
ORDER BY total_reports DESC, c.category_name;
```

JOIN 1

JOINS IN THIS QUERY: 1 (1 INNER)

#	TYPE	TABLES	CONDITION
1	INNER	ITEM i ↔ CATEGORY c	c.category_id = i.category_id

One join to turn category_id into category_name for display and grouping.

HOW TO EXPLAIN IT

- **WHERE** removes individual rows first: only reports dated 29 Sep or later survive.
- **GROUP BY** then collapses the remaining rows into one row per category.
- **COUNT(*)** counts reports in each group. **SUM(CASE ...)** counts lost and found separately, which is conditional aggregation.
- **HAVING** filters whole groups after counting. Electronics disappears: it has 2 reports in total but only 1 dated 29 Sep or later.

SAMPLE OUTPUT

CATEGORY_NAME	TOTAL_REPORTS	LOST	FOUND
Accessories	4	1	3
Books	2	1	1
Documents	2	1	1
Keys	2	1	1

4 rows selected

They may ask: What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before they are grouped and cannot use aggregates. HAVING filters groups after aggregation and can use COUNT, SUM and so on.

Aggregates

GROUP BY

WHERE vs HAVING

Conditional aggregation

Q8a Found items nobody has claimed (outer join)

2 joins

Used on: Admin / statistics report (extra)

Finds found items that still have no claim at all, which are the ones staff should chase.

```
SELECT i.item_id,
       i.item_name,
       c.category_name,
       i.reported_date
FROM   item i
JOIN   category c ON c.category_id = i.category_id
LEFT JOIN claim cl ON cl.item_id = i.item_id
WHERE  i.item_status = 'FOUND'
AND    cl.item_id IS NULL
ORDER BY i.reported_date;
```

JOIN 1

JOIN 2

JOINS IN THIS QUERY: 2 (1 INNER + 1 LEFT OUTER)

#	TYPE	TABLES	CONDITION
1	INNER	ITEM i ↔ CATEGORY c	c.category_id = i.category_id
2	LEFT OUTER	ITEM i → CLAIM cl	cl.item_id = i.item_id

A LEFT JOIN keeps every row from the left table (ITEM) even when CLAIM has nothing to match.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	CATEGORY_NAME	REPORTED_DATE
1011	Steel Water Bottle	Accessories	04-OCT-26
1012	Umbrella	Accessories	06-OCT-26

2 rows selected

HOW TO EXPLAIN IT

- For an item with no claim, every CLAIM column comes back as NULL.
- cl.item_id IS NULL keeps exactly those rows. Joining and then testing for NULL is called an **anti-join**.
- item_status = 'FOUND' limits the result to found reports, since only those can be claimed.

They may ask: What would an INNER JOIN have returned here?

Answer: Only items that do have a claim, which is the opposite of what we want. The outer join is what lets the "no match" rows survive.

LEFT OUTER JOIN

Anti-join

NULL handling

Q8b Same result using a subquery (NOT EXISTS)

1 join

Used on: Admin / statistics report (extra)

The same answer as Q8a written with a correlated subquery instead of an outer join.

```
SELECT i.item_id,
       i.item_name,
       c.category_name,
       i.reported_date
FROM   item i
JOIN   category c ON c.category_id = i.category_id
WHERE  i.item_status = 'FOUND'
AND    NOT EXISTS (SELECT 1
                   FROM    claim cl
                   WHERE   cl.item_id = i.item_id)
ORDER BY i.reported_date;
```

JOIN 1

JOINS IN THIS QUERY: 1 (1 INNER + 1 correlated subquery)

#	TYPE	TABLES	CONDITION
1	INNER	ITEM i ↔ CATEGORY c	c.category_id = i.category_id

The NOT EXISTS subquery is not a join. It is a separate query that runs for each item.

HOW TO EXPLAIN IT

- For each found item, the inner SELECT looks for a claim with the same `item_id`. It refers to `i.item_id` from the outer query, which is why it is called **correlated**.
- `NOT EXISTS` is true only when that inner SELECT finds nothing.
- Q8a and Q8b return identical rows, so you can compare the two styles side by side.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	CATEGORY_NAME	REPORTED_DATE
1011	Steel Water Bottle	Accessories	04-OCT-26
1012	Umbrella	Accessories	06-OCT-26

2 rows selected

They may ask: Why NOT EXISTS rather than NOT IN?

Answer: If the subquery returns even one NULL, NOT IN yields no rows at all. NOT EXISTS is not affected by NULLs.

Correlated subquery

NOT EXISTS

Join vs subquery

Q9 Top 3 reporters

1 join

Used on: Admin / statistics report (extra)

Ranks students by how many items they have reported.

```
SELECT s.student_id,
       s.student_name,
       s.department,
       COUNT(*) AS reports
FROM   student s
JOIN   item i ON i.student_id = s.student_id
GROUP BY s.student_id, s.student_name, s.department
ORDER BY reports DESC, s.student_name
FETCH FIRST 3 ROWS ONLY;
```

JOIN 1

JOINS IN THIS QUERY: 1 (1 INNER)

#	TYPE	TABLES	CONDITION
1	INNER	STUDENT s ↔ ITEM i	i.student_id = s.student_id

One join so each report can be counted against the student who made it.

HOW TO EXPLAIN IT

- Joining then grouping by student gives one row per student; `COUNT(*)` is the number of reports.
- `ORDER BY reports DESC` puts the busiest reporter first. The second sort key, the name, makes ties come out in a fixed order.
- `FETCH FIRST 3 ROWS ONLY` keeps the top three (Oracle 12c or newer).
- Four students have 2 reports, so only two of them fit in places 2 and 3. `FETCH FIRST 3 ROWS WITH TIES` would include everyone tied for third place.

SAMPLE OUTPUT

STUDENT_ID	STUDENT_NAME	DEPARTMENT	REPORTS
5002	Diya Patel	ECE	3
5001	Aarav Sharma	CSE	2
5005	Kabir Singh	CIVIL	2

3 rows selected

They may ask: How would you write this before Oracle 12c?

Answer: Put the ordered query in a subquery in the FROM clause and filter the outer query with `WHERE ROWNUM <= 3`.

JOIN + GROUP BY

Top-N

Tie handling

Used on: Admin / statistics report (extra)

One row per item with its category, location, reporter, number of claims and number of images. It touches every entity in the ER diagram.

```
SELECT i.item_id,
       i.item_name,
       i.item_status,
       c.category_name,
       l.location_name,
       s.student_name AS reported_by,
       COUNT(DISTINCT cl.student_id) AS claims,
       COUNT(DISTINCT im.image_no) AS images
FROM   item i
JOIN   category c ON c.category_id = i.category_id
JOIN   location l ON l.location_id = i.location_id
JOIN   student s ON s.student_id = i.student_id
LEFT JOIN claim cl ON cl.item_id = i.item_id
LEFT JOIN item_image im ON im.item_id = i.item_id
GROUP BY i.item_id, i.item_name, i.item_status,
         c.category_name, l.location_name, s.student_name
ORDER BY i.item_id;
```

JOIN 1

JOIN 2

JOIN 3

JOIN 4

JOIN 5

JOINS IN THIS QUERY: 5 (3 INNER + 2 LEFT OUTER)

#	TYPE	TABLES	CONDITION
1	INNER	ITEM i ↔ CATEGORY c	c.category_id = i.category_id
2	INNER	ITEM i ↔ LOCATION l	l.location_id = i.location_id
3	INNER	ITEM i ↔ STUDENT s	s.student_id = i.student_id
4	LEFT OUTER	ITEM i → CLAIM cl	cl.item_id = i.item_id
5	LEFT OUTER	ITEM i → ITEM_IMAGE im	im.item_id = i.item_id

Six tables, five joins. The two outer joins keep items that have no claims or no images, which then show 0.

SAMPLE OUTPUT

ITEM_ID	ITEM_NAME	ITEM_STATUS	CATEGORY_NAME	LOCATION_NAME	REPORTED_BY	CLAIMS	IMAGES
1001	Black Earbuds	LOST	Electronics	Library	Aarav Sharma	0	0
1002	ID Card	LOST	Documents	Main Block	Diya Patel	0	0
1003	Bike Keys	LOST	Keys	Canteen	Rohan Mehta	0	0
1004	Black Wallet	LOST	Accessories	Parking Area	Sneha Iyer	0	0
1005	DBMS Textbook	LOST	Books	Library	Kabir Singh	0	0
1006	Black Earbuds	FOUND	Electronics	Library	Meera Nair	1	2
1007	ID Card	FOUND	Documents	Main Block	Aarav Sharma	1	1
1008	Bike Keys	FOUND	Keys	Library	Diya Patel	2	0
1009	Brown Wallet	FOUND	Accessories	Canteen	Meera Nair	1	0
1010	Notebook	FOUND	Books	Parking Area	Rohan Mehta	1	1
1011	Steel Water Bottle	FOUND	Accessories	Main Block	Kabir Singh	0	0
1012	Umbrella	FOUND	Accessories	Main Block	Diya Patel	0	0

12 rows selected

They may ask: Why COUNT(DISTINCT ...) here?

Answer: Because the claim and image joins multiply each other's rows. DISTINCT counts each claim and each image once, so item 1006 shows 1 claim and 2 images instead of 2 and 2.

Six-table join

Outer joins

Row multiplication

COUNT(DISTINCT)

Common viva questions

JOINS

- **INNER vs LEFT OUTER join:** INNER keeps only rows that match on both sides. LEFT OUTER keeps every row of the left table and fills the right side with NULL when nothing matches (Q8a, Q10).
- **Self-join:** a table joined to itself under two aliases. Q3 pairs ITEM with ITEM to compare a lost report with a found one.
- **Why join at all?** Normalised tables keep each fact once. Joins reassemble them for display (Q2, Q5).
- **Join vs subquery:** Q8a and Q8b give the same rows. A join is usually easier to read for combining columns; a subquery suits yes/no questions such as "does a claim exist?".

DESIGN

- **Primary key vs foreign key:** the primary key identifies a row; a foreign key points at another table's primary key and keeps the data consistent.
- **Strong vs weak entity:** CLAIM and ITEM_IMAGE cannot exist without an ITEM, so their keys include item_id.
- **Why separate CATEGORY and LOCATION tables?** To avoid repeating names and to keep updates in one place (3NF).

REPRODUCING THE OUTPUT

The schema, sample data and every query above are in `docs/campus_finder_queries.sql`. Run it in SQL*Plus and the results match the sample output shown here (dates print as DD-MON-YY).